

DS 2002 Artificial Intelligence

Possible Project

Spring 2024

Instructions:

Choose one of the mini-project from the list of projects given in this document and complete it before 5th May

Following rules will be enforced for this Project

- You might work in a group of at most Three students (Across-section group are allowed)
- Submit you group information on Google Classroom before Saturday
- Your group will submit a detailed report explaining your implementation details

Please remember that PLAGIARISM is INTOLERABLE and anyone found involved in it will get -5 marks (i.e. 100% penalty) in this assignment.

DS 2002 Artificial Intelligence

Possible Project

Spring 2024

Chess Player

Game of chess has been a classic AI problem and these days there are several excellent open source implementations of automatic chess players, like GNU chess, are available on line. In this assignment we are going to make yet another FAST-chess player that will play chess using some well-established strategies along with **MINIMAX** to compute moves for a chess player.

The program will be implemented in Python and must have the following components at minimum

1. Your own Evaluation function based on well researched evaluation functions available in literature
2. An optimal representation of the game state and a correct implementation of the minimax with pruning algorithm for selecting a move
3. Allow the user to specify the depth of the game tree at the beginning of the game
4. A basic interface for playing the game

DS 2002 Artificial Intelligence

Possible Project

Spring 2024

Make Squares also called Dots and Boxes**You can play this game online at**<https://www.math.ucla.edu/~tom/Games/dots&boxes.html>**Game Description**

In this programming assignment we are going to implement only the basic MINIMAX algorithm for creating a playing agents for the game "Make Squares"

The "Make Squares" is a zero-sum two-player game that is played on an grid of $M \times N$ squares or (i.e. $(M+1) \times (N+1)$ dots representing corners of the squares) using the following rules.

1. The players take turns and in each turn a player must connect one dot to one of its unconnected neighboring dot.
2. The player who connects the last unconnected dots of a square is awarded one point and in such a case he must make an additional turn.

The game is won by the player who wins most boxes out of the possible $M \times N$ boxes.

DS 2002 Artificial Intelligence

Possible Project

Spring 2024

Exam Schedule Generation Using Hill Climbing Search

Manually generating a Mid-term exam schedule for NUCES-FAST is an involved task as a diverse set of constraints must be enforced while creating the schedule. In this mini-Project we are going to use the local search based optimization to solve the scheduling problem.

Major inputs (for creating an exam schedule of m courses) to your implementation will include

- A file named "**registration.data**" that contains course registration status in the form of a 2D $m \times m$ array with space separated entries stored in row major order. Entry a_{ij} of the array will specify the number of students taking both the courses i and j .
- A file named "**capacity.room**" containing a space separated list of room capacities available for scheduling.
- A file named "**general.info**" containing a number specifying total exam days followed by a single number giving the exam slots per day for each room.

Your main job in this assignment includes the design and implementation of

- An efficient representation of a solution
- A method to create neighbors of a solution
- Implement hill climbing search to find an optimal solution
- A main that uses the optimization method to repeatedly start the search at multiple points and find one best solution

As any reasonable exam schedule will have all of the following properties therefore your fitness function must consider all of these properties for computing fitness of a chromosome.

1. All exams must be scheduled within the given number of days.
2. Total students taking exam in one given slots must be less than the total room capacity.
3. Number students having two exams in one given slot must be minimized.
4. Number of students having exams on two consecutive slots must be minimized.
5. Student with more than two exams in a single time slot must be minimized
6. Student with more than two exams in a day must be minimized.
7. Student with more than two exams in consecutive slots must be minimized
8. Not even a single student can have more than three exams in one given day.

DS 2002 Artificial Intelligence

Possible Project

Spring 2024

Rubik's Cube Solver

The Rubik's Cube is an important test domain for heuristic search that has 10^{19} possible states, making it impossible to store in memory. A single move on a Rubik's Cube can be defined as a quarter turn (90°) of any face either clockwise or anticlockwise.



In this mini-project you must use bi-directional search (A*) to find moves that will take as input **initial** and **final** configurations of the cube and will output **minimum number of moves** (i.e. 90° turns of faces) needed to transform the initial state into the final state.